Survey on Spectral Geometry as Positional Encoding in Graph Transformers

Gabriele Moisè

Università degli Studi di Milano-Bicocca

Outline

Motivation

Spectral Attention Network

Spectral Positional Encodings

Obstacles

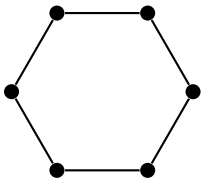
Methods

Conclusion

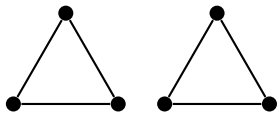
Expressive power of message passing

- Message-passing GNNs are at most as expressive as the 1-Weisfeiler–Leman (1-WL) graph isomorphism test (Xu et al., 2019).
- 1-WL iteratively refines node colors and compares the resulting histograms:
  - different histograms $\Rightarrow$ graphs are *not* isomorphic;
  - same histogram $\Rightarrow$ graphs *may* be isomorphic.
- If 1-WL maps two graphs to the same histogram, an MPNN cannot distinguish them.

A failure case of 1-WL



C_6

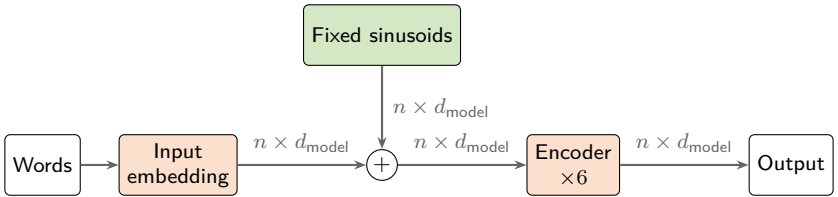


$C_3 \cup C_3$

Every node has degree 2 in both graphs, so 1-WL (and any MPNN) cannot tell them apart.

Positional encodings inject the structural information that message passing misses.

The starting point: Attention Is All You Need

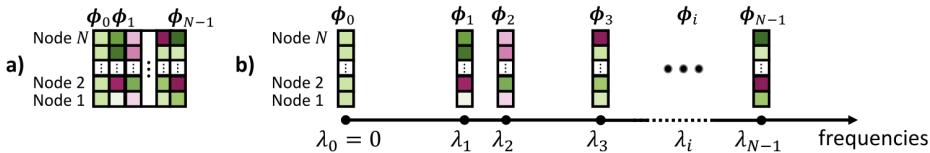


$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right), \quad PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$$
$$(d_{\text{model}} = 512)$$

- A sequence has a natural order, so pos is an integer and the frequencies can be fixed in advance (Vaswani et al., 2017).
- PE and embedding have the same size, so they can be summed.
- A graph has no pos, and each graph has its own frequencies. What replaces the sinusoids?

From sinusoids to Laplacian eigenfunctions

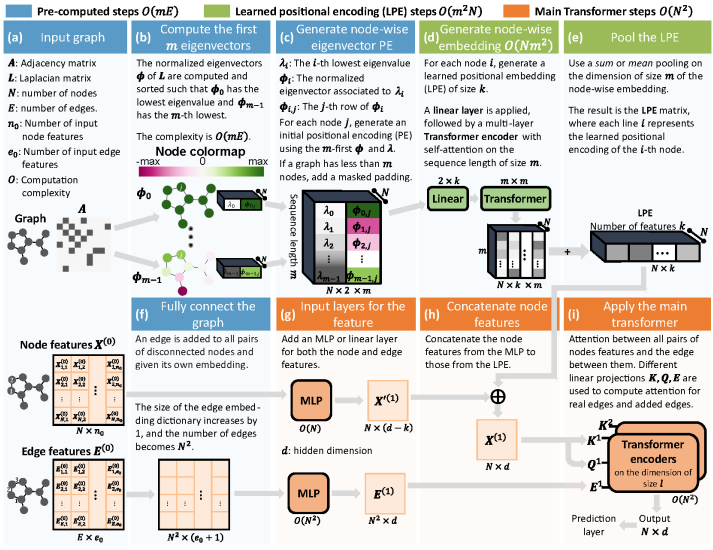
- In Euclidean space, the eigenfunctions of the Laplacian *are* sinusoids, with eigenvalues given by the squared frequencies.
- On a graph, the natural equivalent is the eigenvectors ϕ_i of L (Kreuzer et al., 2021).



Kreuzer et al. (2021, Fig. 2): (a) eigenvectors as a matrix, (b) eigenvectors placed on the axis of eigenvalues.

Key idea: (b) is a sequence, with non-uniform positions given by λ_i. Sequences are what Transformers process.

SAN: two Transformers

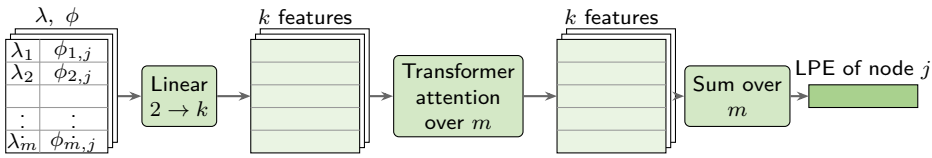


LPE Transformer attention over the m eigenpairs of each node

Main Transformer attention over all pairs of the N nodes

Figure: Kreuzer et al. (2021, Fig. 1).

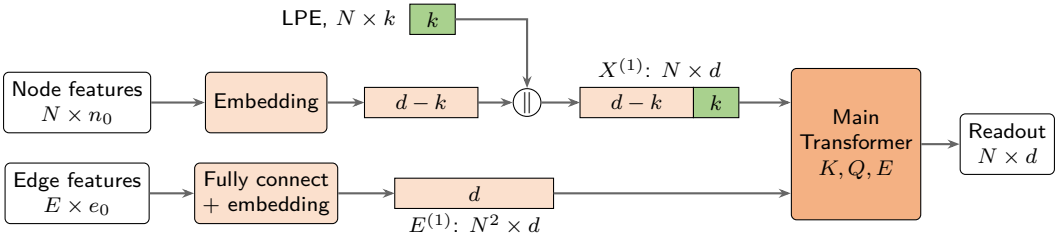
Learned Positional Encoding: following the dimensions



Node j:	$m \times 2$	$m \times k$	$m \times k$	$1 \times k$
All:	$m \times N \times 2$	$m \times N \times k$	$m \times N \times k$	$N \times k$
ZINC:	$10 \times N \times 2$	$10 \times N \times 8$	$10 \times N \times 8$	$N \times 8$

- m is the sequence length (eigenpairs); N acts as the batch (the same network runs on every node); k is the hidden size.
- No separate PE: λ_i is an input channel, so it is the position.
- If $N < m$, the missing eigenpairs are masked. Cost: $O(m^2 N)$.

How the LPE enters the embedding



- The node embedding is shortened by k to make room for the LPE (ZINC: $48 + 8 = 56$).
- Unlike Vaswani et al. (2017), the PE is concatenated, not summed: it gets its own k dimensions.

```
self.embedding_h = nn.Embedding(num_atom_type, GT_hidden_dim-LPE_dim)
h = torch.cat((h, PosEnc), 1)   # N x (d-k) + N x k = N x d
```

Same blueprint as Attention Is All You Need

	Attention Is All You Need	**SAN: LPE**	**SAN: main**
Sequence elements	n words	m eigenpairs	N nodes
Batch dimension	sentences	nodes	graphs
Hidden size	$d_{\text{model}} = 512$	k (ZINC: 8)	d (ZINC: 56)
Position	integer pos	eigenvalue λ_i	LPE vector
Position source	fixed sinusoids	given by the spectrum	learned
Combination	sum	concatenation	concatenation
Attention over	all words	all eigenpairs of a node	all node pairs
Decoder	yes	no	no

- The LPE is a standard Transformer encoder, applied along the frequency axis rather than along a sentence.
- Its output plays the role of the sinusoids for the main Transformer: a learned replacement for a fixed formula.

Attention: from sequences to graphs

Attention Is All You Need

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

SAN: the same scaled dot product, with edge features and two sets of projections

$$\hat{w}_{ij} = \frac{Qh_i \circ Kh_j \circ Ee_{ij}}{\sqrt{d_k}}, \qquad w_{ij} = \begin{cases} \frac{1}{1+\gamma}\,\text{softmax}(\sum_{d_k} \hat{w}_{ij}) & \text{real edge} \\ \frac{\gamma}{1+\gamma}\,\text{softmax}(\sum_{d_k} \hat{w}_{ij}) & \text{added edge} \end{cases}$$

- Real and added edges have separate Q, K, E.
- γ tunes the bias: $\gamma = 0$ is sparse attention, while large γ favors full attention.
- The graph structure enters twice: through the LPE and through the real edges.

Excursus: PE on nodes vs. on edges

	Node LPE	**Edge LPE**
Type of position	Absolute	Relative
Input per eigenpair	$\lambda_i,\ \phi_{i,j}$	$\lambda_i,\ \lvert\phi_{i,j_1} - \phi_{i,j_2}\rvert,\ \phi_{i,j_1}\phi_{i,j_2}$
Tensor (batch = nodes or edges)	$m \times N \times 2$	$m \times N^2 \times 3$
Linear layer	$2 \to k$	$3 \to k$
Output	$N \times k$	$N^2 \times k$
Concatenated to	node embedding $(d - k)$	edge embedding $(d - k)$
Sign-invariant	No	Yes
Cost (fully connected graph)	$O(m^2 N)$	$O(m^2 N^2)$

- Why invariant: $|(-a) - (-b)| = |a - b|$ and $(-a)(-b) = ab$. Only the relative sign between two nodes survives.
- Intuition from the paper: like potential energy, position is meaningful only as a difference.
- The edge LPE is cleaner but expensive: all the repository's optimized configurations use the node LPE, which needs a sign fix.

Paper vs. code: the sign flip

Paper, §3.1: *"we randomly flip the sign of the pre-computed eigenvectors during training"*.

```
#random sign flipping
sign_flip = torch.rand(batch_EigVecs.size(1)).to(device)
sign_flip[sign_flip>=0.5] = 1.0; sign_flip[sign_flip<0.5] = -1.0
batch_EigVecs = batch_EigVecs * sign_flip.unsqueeze(0)  # missing in ZINC and SBMs
batch_scores = model.forward(batch_graphs, batch_x, batch_e, batch_EigVecs, batch_EigVals)
```

	ZINC	PATTERN	CLUSTER	MOLHIV	MOLPCBA
Optimized config uses	node LPE	node LPE	node LPE	node LPE	node LPE
Flip computed	✓	✓	✓	✓	✓
Flip applied	–	–	–	✓	✓

In the public code, 3 of 5 benchmarks train the node LPE with no sign augmentation, in every commit, despite what the paper states.

github.com/DevinKreuzer/SAN, commit 6e38329: `train_ZINC_graph_regression.py`, `train_SBMs_node_classification.py` vs. `train_molhiv.py`, `train_molpcba.py`. The public code may not be exactly what produced the published results.

Laplacian eigenvectors as positions

Normalized graph Laplacian:

$$L = I - D^{-1/2} A D^{-1/2} = V \Lambda V^\top$$

- Use the first k eigenvectors $V_{:,1:k} \in \mathbb{R}^{n \times k}$ as node positions (Dwivedi and Bresson, 2020).
- Generalizes sinusoidal encodings: eigenvectors of the path graph are Fourier modes.
- But eigenvectors are not unique, and this is the source of most obstacles.

Obstacle 1: Sign ambiguity

- If v is an eigenvector, so is $-v$; the solver returns either one depending on implementation details.
- A model that is not sign-invariant sees the same graph as different inputs: parameters oscillate trying to align fluctuating features, which hinders convergence.
- **Random sign flipping** as augmentation (Dwivedi and Bresson, 2020; Kreuzer et al., 2021):
  - does not solve the problem mathematically; the ambiguity is not granted to be solved
  - with k eigenvectors there are 2^k sign combinations, which slows training exponentially;
  - spends model capacity on learning an invariance.

Obstacle 2: Basis ambiguity

- Symmetric graphs often have eigenvalues with multiplicity $d > 1$.
- With distinct eigenvalues, each unit eigenvector is unique up to sign.
- With multiplicity d, any rotation or reflection of an orthonormal basis is still a valid basis: infinitely many choices.
- Consequence: overfitting. The network may learn the bases seen in training, but fails on a similar graph whose solver returns a basis it never saw.

Requirement

Changing the basis of the input eigenvectors must not change the output.

Basis ambiguity, geometrically

- Let λ_i have multiplicity d_i with eigenvectors $V_i \in \mathbb{R}^{n \times d_i}$. For any $O_i \in \mathrm{O}(d_i)$, $V_i O_i$ is still an orthonormal basis of the eigenspace.
- Collecting all eigenspaces gives the symmetry group

$$\mathrm{O}(\lambda) = \bigoplus_i \mathrm{O}(d_i) = \Big\{ \bigoplus_i O_i : O_i \in \mathrm{O}(d_i) \Big\}.$$

- Both (V, λ) and (VQ, λ), $Q \in \mathrm{O}(\lambda)$, are valid eigendecompositions.

Basis invariance

$$f(V, \lambda) = f(VQ, \lambda) \qquad \forall\, Q \in \mathrm{O}(\lambda)$$

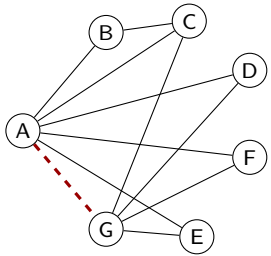
Obstacle 3: Limits of invariant architectures

- **SignNet / BasisNet** (Lim et al., 2023) build the invariance into the architecture, e.g. SignNet uses
$$\phi(v) + \phi(-v).$$
- This is a strong inductive bias:
  - restricts the class of learnable functions, which limits expressivity;
  - increases training time, possibly to reach a suboptimal solution.
- They have the right symmetries, but ignore that two similar but distinct Laplacians can produce completely different eigenspaces.

Obstacle 4: Instability to perturbations

- **Stability**: a small change of the Laplacian should give a small change of the positional encodings. This generalizes basis invariance.
- If eigenvalues are close (small spectral gap), small perturbations can produce completely different eigenvectors (Wang et al., 2022, Lemma 3.4).
- Large, complex graphs (e.g. social networks) often have nearly coinciding eigenvalues, so eigenspaces mix.
- Models that consume eigenvectors inherit this instability: similar graphs (e.g. one edge removed) can map to distant regions of the latent space, causing poor generalization and low robustness to noise.

Obstacle 4: Removing a single edge



Example graph from Liang et al. (2025, Fig. 4); dashed edge removed.

Degrees	A	B	C	D	E	F	G
Before	6	2	3	2	2	2	5
After	5	2	3	2	2	2	4

Adj. eigenvalues	λ_1	λ_2	λ_3	λ_4	λ_5	λ_6	λ_7
Before	3.60	0.85	0	0	-0.53	-1.55	-2.38
After	3.10	0.94	0	0	0	-1.20	-2.84
$\lvert\Delta\lambda\rvert$	0.50	0.09	0	0	0.53	0.34	0.46

- Five of the seven eigenvalues change, yet each moves by at most $\|\Delta A\|_2 = 1$ (Weyl): the eigen*values* stay controlled.
- The multiplicity of $\lambda = 0$ jumps from 2 to 3: the solver may return any basis of the larger eigenspace, so the computed eigen*vectors* need not resemble the previous ones.

Obstacle 5: Stability vs. expressivity

- Sign-invariant methods (Dwivedi and Bresson, 2020; Kreuzer et al., 2021; Lim et al., 2023) do not guarantee "same Laplacian, same encoding" when eigenvalues have multiplicity.
- **BasisNet** is basis-invariant but unstable: it applies different networks to different eigenspaces (hard partitioning).
- **PEG** (Wang et al., 2022) is provably stable, but processes all eigenspaces homogeneously, which loses expressive power (e.g. weaker results on molecular regression, Rampášek et al., 2022).

Open question

Can a positional encoding be stable *and* expressive?

Overview of approaches

Method	Encoding	Sign inv.	Basis inv.	Stable
LapPE (Dwivedi and Bresson, 2020)	Laplacian eigenvectors	aug.	–	–
SAN (Kreuzer et al., 2021)	Learned spectral attention	aug.	–	–
SignNet (Lim et al., 2023)	$\phi(v) + \phi(-v)$	✓	–	–
BasisNet (Lim et al., 2023)	Per-eigenspace networks	✓	✓	–
PEG (Wang et al., 2022)	Eigenvector distances	✓	✓	✓
SPE (Huang et al., 2024)	Soft eigenspace weighting	✓	✓	✓

aug. = invariance only encouraged through random sign flipping. SAN: node LPE; its public code applies the flip only on MOLHIV and MOLPCBA.

Toward a solution: Stable and Expressive PE

- SPE (Huang et al., 2024) replaces hard eigenspace partitioning with a soft, eigenvalue-dependent weighting:

$$\mathrm{SPE}(V, \lambda) = \rho([\, V \,\mathrm{diag}(\phi_1(\lambda))V^\top, \ldots, V \,\mathrm{diag}(\phi_m(\lambda))V^\top])$$

- Each term is basis-invariant, since $V \,\mathrm{diag}(\cdot)V^\top$ does not depend on the chosen basis.
- With Lipschitz ϕ_ℓ, close eigenvalues are treated similarly, giving provable stability without discarding eigenspace information.

Beyond the Laplacian: NAPE

NAPE (Ajayi et al., 2024) never computes eigenvectors, so it has no sign or basis ambiguity: it learns a node numbering for the sinusoidal PE of Vaswani et al. (2017).

$$Z_i = \sigma(\gamma P_i + \epsilon) \in \{0,1\}^m \longrightarrow t = \mathrm{Dec}(Z_i) \longrightarrow \mathrm{Pos}(t)$$

- Hamming distance on the hypercube mimics graph closeness:

$$\mathcal{L} = \mathcal{L}_{adj} + \lambda_1 \mathcal{L}_{deg_dist} + \lambda_2 \mathcal{L}_{deg}$$

- Eigenvectors identify clusters, and so do BFS-like random walks. With negative sampling, the cost is $O(|V|^2)$ if $l \cdot r \cdot k \leq |V|$.

	LapPE		scaled-NAPE	
Dataset	Score	Time	Score	Time
WikiCS	77.3	5.25 m	77.7	2.50 m
PubMed	85.3	22.0 m	84.5	1.60 m
OGBL-collab	out of memory		50.4	767 m

Accuracy, or Hits@50 for OGBL-collab (baseline 53.9).
r walks of length l per node, k negative samples.

Limits: works on a single graph at a time; encodings are not always unique (under 0.5% of nodes shared one); needs a hyperparameter search; falls below the baseline on multi-graphs; does not address stability.

Takeaways

- Spectral encodings overcome the 1-WL limit by giving graph transformers a notion of position.
- Eigenvectors are not unique: sign and basis ambiguities must be handled by design, not by augmentation.
- Invariance alone is not enough: encodings must also be stable under small perturbations of the graph.
- The central trade-off is stability vs. expressivity; SPE is a step toward achieving both.

References I

Olayinka Ajayi, Hongkai Wen, and Tanaya Guha. NAPE: Numbering as a position encoding in graphs. *IEEE Access*, 12:166200–166210, 2024.

Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699*, 2020.

Yinan Huang, William Lu, Joshua Robinson, Yu Yang, Muhan Zhang, Stefanie Jegelka, and Pan Li. On the stability of expressive positional encodings for graphs. In *International Conference on Learning Representations*, 2024.

Devin Kreuzer, Dominique Beaini, William L. Hamilton, Vincent Létourneau, and Prudencio Tossou. Rethinking graph transformers with spectral attention. In *Advances in Neural Information Processing Systems*, 2021.

Bin Liang, Shiwei Chen, Lin Gui, Hui Wang, Yue Yu, Ruifeng Xu, and Kam-Fai Wong. Centrality-guided pre-training for graph. In *International Conference on Learning Representations*, 2025.

Derek Lim, Joshua Robinson, Lingxiao Zhao, Tess Smidt, Suvrit Sra, Haggai Maron, and Stefanie Jegelka. Sign and basis invariant networks for spectral graph representation learning. In *International Conference on Learning Representations*, 2023.

References II

Ladislav Rampášek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. In *Advances in Neural Information Processing Systems*, 2022.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Haorui Wang, Haoteng Yin, Muhan Zhang, and Pan Li. Equivariant and stable positional encoding for more powerful graph neural networks. In *International Conference on Learning Representations*, 2022.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.